



Programación de Dispositivos Móviles

UNIDAD 1 | INTRODUCCIÓN A LAS TECNOLOGÍAS MÓVILES

ACTIVIDAD 3

ABRAHAM ESPINOSA CARBAJAL



Contenido

Descripción de Actividad	2
Desarrollo de Actividad.....	3
Conclusión.....	7
Referencias	8



Descripción de Actividad

1. Crear tu primer activity
2. En lenguaje de programación JAVA dentro de Android Studio, crea una app vacía.
3. Crear un arreglo con 1 millón de elementos enteros de manera aleatoria.
4. Ordenar los elementos.
5. Medir el tiempo de inicio a fin.
6. Imprimir en el logcat los resultados obtenidos.

Subir actividad en el repositorio de github creado por el alumno.

1. [android_2027_m1_03.pdf](#)
2. [código fuente.](#)



Desarrollo de Actividad

Creación de mi primer Activity

Iniciamos seleccionando Empty Activity

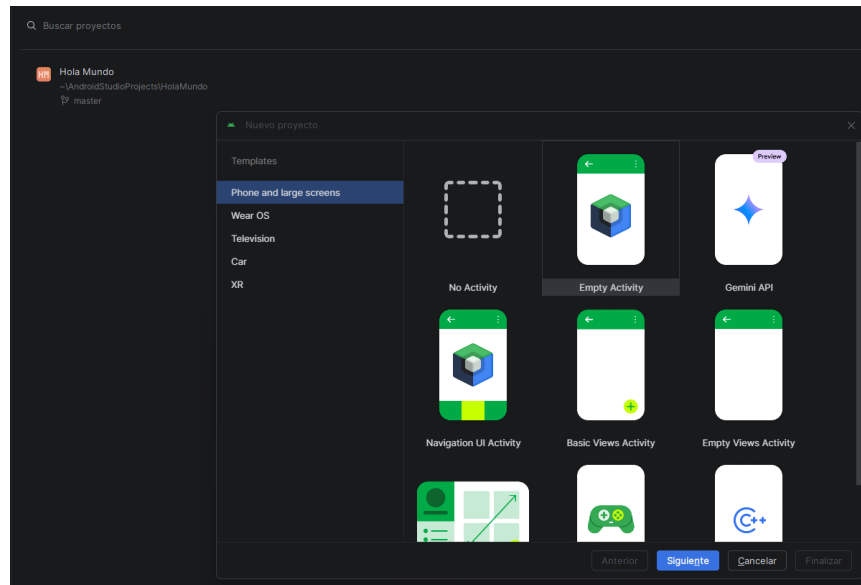


Ilustración 1. Creación de código

Damos nombre al proyecto “My Application”. Asignamos carpeta de almacenamiento del proyecto.

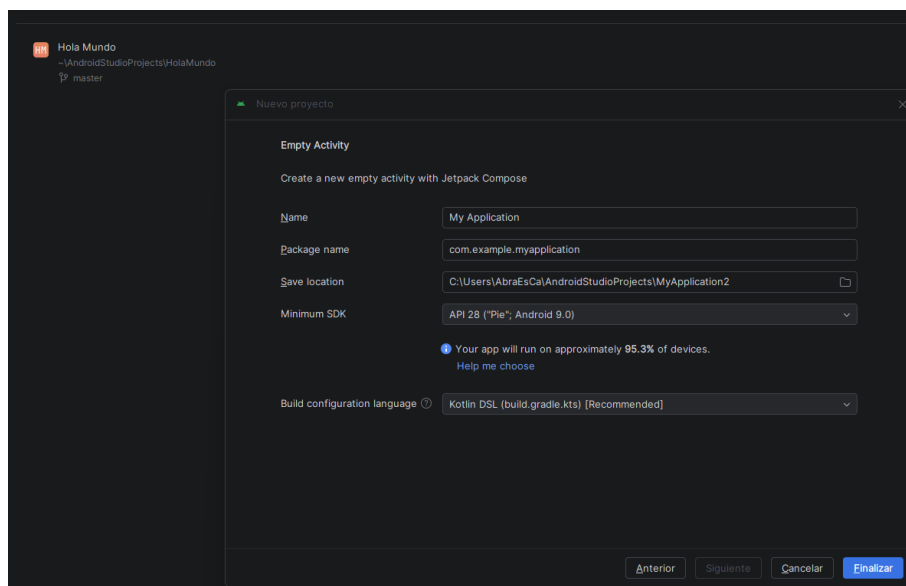


Ilustración 2. Logcast del Texto

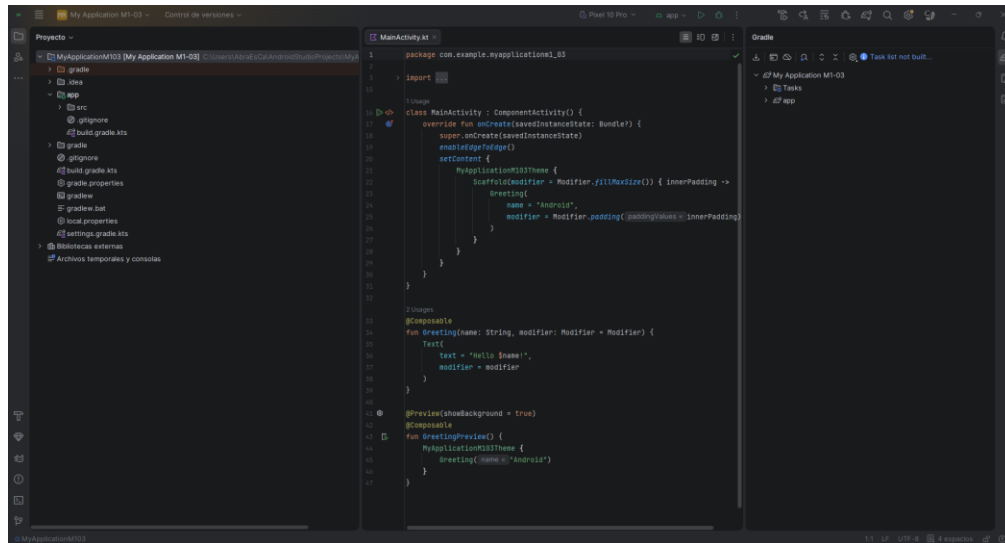


Ilustración 3. Creación de primer activity

Creación de app vacía:

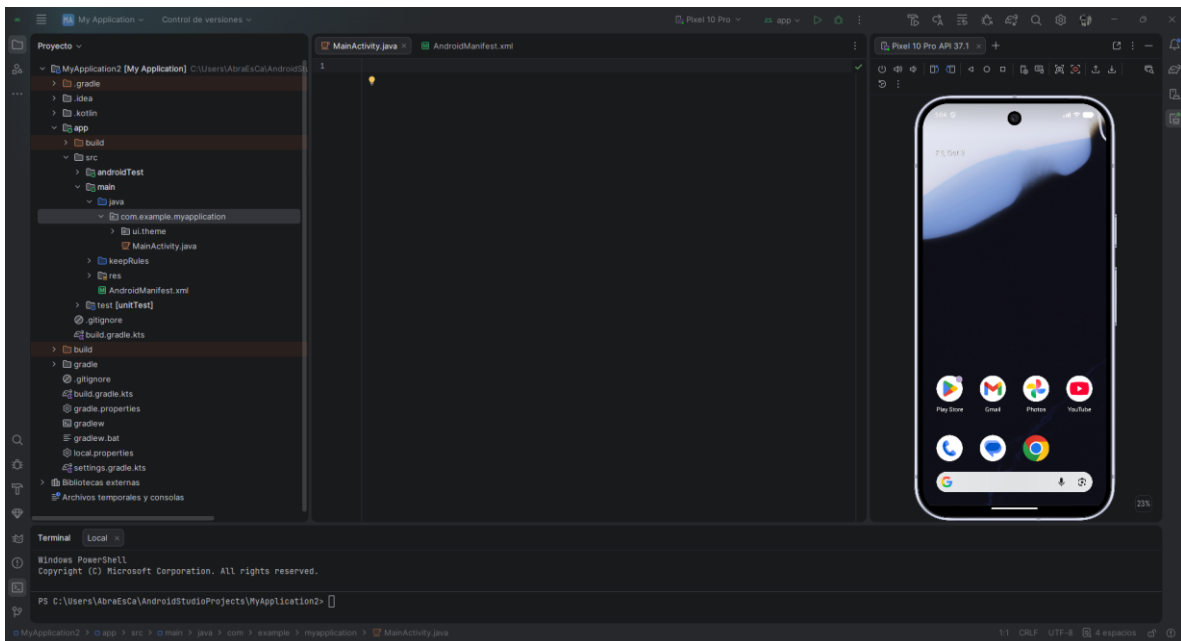


Ilustración 4. App vacía



Creación de arreglo con 1 millón de elementos

Crea un arreglo de un millón de números enteros

```
8 usages
12     private static final String TAG = "ORDENAMIENTO";
13
14     @Override
15     protected void onCreate(Bundle savedInstanceState) {
16         super.onCreate(savedInstanceState);
17
18         // Arreglo de 1,000,000 de elementos
19         int[] numeros = new int[1_000_000];
```

Ilustración 5. `Int [1_000_000]`.

Se genera números aleatorios y los almacena en cada posición del arreglo

```
3
4     // Generador de números aleatorios
5     Random random = new Random();
```

Ilustración 6. Generar número aleatorio

`System.nanoTime()` permite medir con bastante precisión el tiempo transcurrido entre el inicio y el final del proceso de ordenamiento.

```
28
29     // Iniciar medición del tiempo
30     long tiempoInicio = System.nanoTime();
31
32     // Ordenar los elementos
33     Arrays.sort(numeros);
34
35     // Finalizar medición
36     long tiempoFin = System.nanoTime();
```

Ilustración 7. Medir y ordenar.

Tiempo empleado en nanosegundos.

```
38     // Calcular tiempo transcurrido
39     long tiempoTotal = tiempoFin - tiempoInicio;
40
41     // Convertir a milisegundos
42     double tiempoMilisegundos = tiempoTotal / 1_000_000.0;
```

Ilustración 8. Medición del tiempo



Logcat

Indicaciones para mostrar resultados en Logcat.

```
// Mostrar resultados en Logcat
Log.d(TAG, msg: "=====");
Log.d(TAG, msg: "ORDENAMIENTO DE 1 MILLÓN DE ELEMENTOS");
Log.d(TAG, msg: "=====");
Log.d(TAG, msg: "Cantidad de elementos: " + numeros.length);
Log.d(TAG, msg: "Primer elemento: " + numeros[0]);
Log.d(TAG, msg: "Último elemento: " + numeros[numeros.length - 1]);
Log.d(TAG, msg: "Tiempo de ordenamiento: " + tiempoMilisegundos + " ms");
Log.d(TAG, msg: "=====");
}
```

Ilustración 9. Indicaciones de mostrar resultados en Logcat

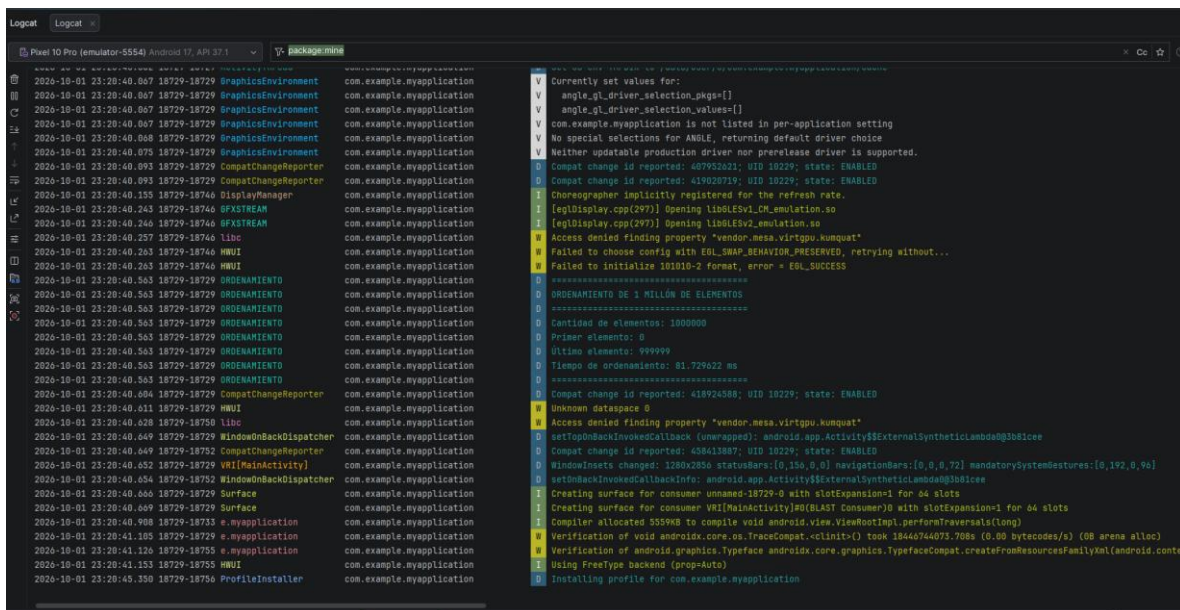


Ilustración 10. Resultados en Logcat.

Link del repositorio del código: <https://github.com/AbraEsCa/AppOrdenamiento>

Repositorio de la actividad en PDF: <https://github.com/AbraEsCa/abraesca>



Conclusión

Esta actividad permite implementar en Java la creación y ordenamiento de un arreglo de un millón de números enteros aleatorios, así como medir el tiempo de ejecución del proceso. Con ello, es posible observar de manera práctica el funcionamiento de un algoritmo de ordenamiento y comprobar cómo el tiempo de procesamiento puede evaluarse mediante herramientas de medición y el Logcat de Android Studio.



Referencias

Get started (sf). *Comienza a usar Android*. Developers Android, de:
<https://developer.android.com/get-started/overview?hl=es-419>